

Predicting User Sentiment in Software Issue Reports

ST2MLE — Machine Learning for IT Engineers · Project Report

1. Project overview

We build and evaluate a machine-learning pipeline that classifies the sentiment of user-reported software issues into **negative**, **neutral** and **positive**. The study compares **three text-vectorisation strategies** (Bag-of-Words, TF-IDF, Word2Vec — plus an optional BERT path) against **five classifiers** (Naive Bayes, Decision Tree, Random Forest, AdaBoost, Logistic Regression), each hyper-parameter-tuned with 5-fold cross-validation. We then study, in dedicated experiments, class-imbalance handling, dimensionality reduction, decision-tree pruning and early stopping.

All numbers in this report are produced by `python main.py` and are fully reproducible (single global seed `RANDOM_STATE = 42`).

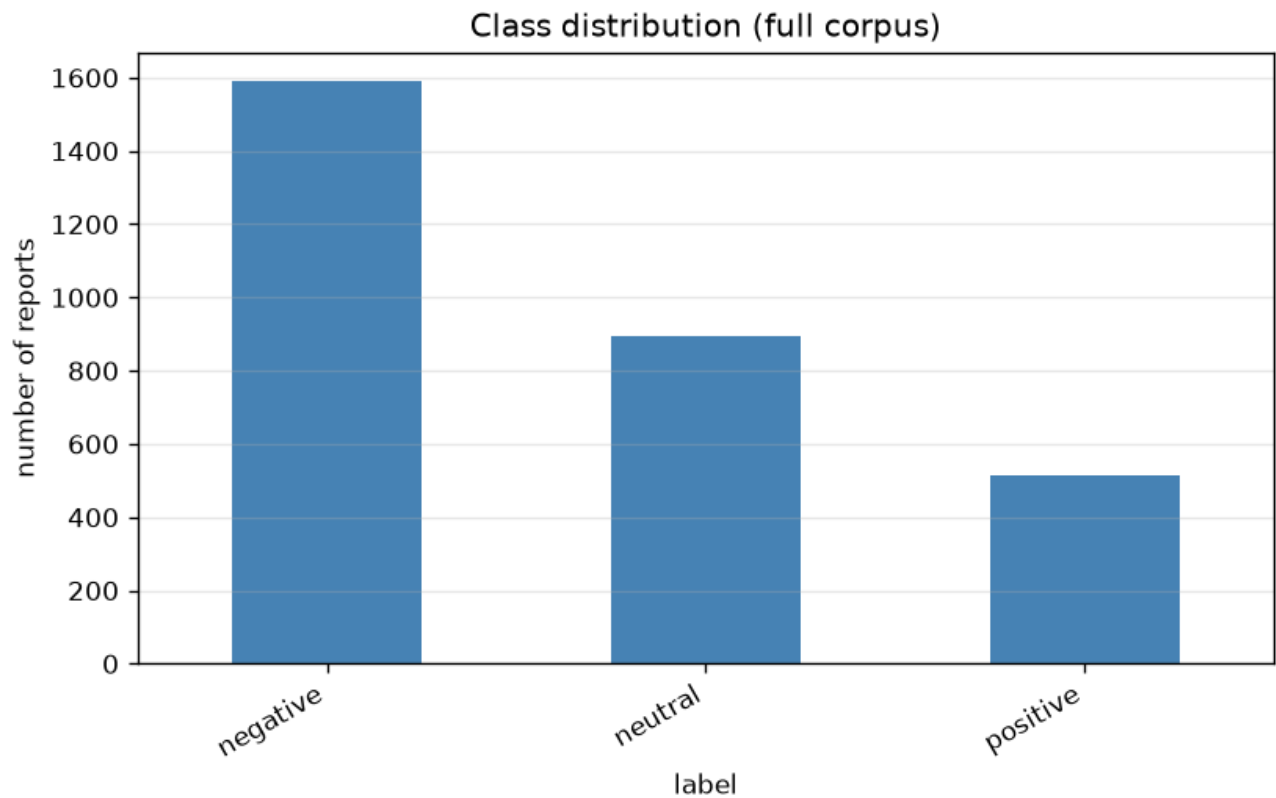
2. Dataset description

2.1 Source and rationale

Ready-to-use sentiment-labelled issue corpora require API credentials or downloads from hosts unreachable in our sandbox, so we use a **reproducible synthetic generator** ([src/data_generation.py](#)) that emulates the linguistic structure of real issue trackers. Each report is assembled from realistic vocabulary banks (components, versions, actions) and class-specific sentiment clauses, openers and closing remarks.

2.2 Composition (3 000 reports)

Class	Meaning	Count	Share
negative	crashes, bugs, frustration	1 590	53.0 %
neutral	questions, feature requests, factual notes	896	29.9 %
positive	praise, thanks, "works great now"	514	17.1 %



The imbalance is deliberate (issue trackers are dominated by complaints) and motivates the resampling experiment in §6.

2.3 What makes the task non-trivial

A naïve synthetic dataset would be perfectly keyword-separable and every model would saturate at the same score. We inject three realistic difficulties:

1. **Shared software vocabulary** across all classes — the model must learn sentiment, not topic.
2. **Ambiguity & mixed signals** — 22 % of reports use a tone-neutral core clause ("does the job, more or less"), and 15 % borrow an opener/closing remark from another class, producing genuinely mixed-signal text.
3. **Label noise** — 8 % of labels are randomly flipped, emulating annotation disagreement and capping achievable accuracy below 100 %.

Together these lower the achievable ceiling to a realistic $\approx$ **0.88 macro-F1** and create meaningful spread between models.

3. ML pipeline overview

```
raw text → preprocess → vectorise → [balance] → [reduce] → classify → evaluate
```

3.1 Preprocessing ([src/preprocessing.py](#))

Lower-casing → URL/HTML/punctuation stripping → tokenisation → stop-word removal → WordNet lemmatisation. Two sentiment-aware choices:

- **Negations are kept** (`not` , `no` , `never` ...): they flip sentiment, so removing them as ordinary stop-words would destroy signal.
- **Two-pass lemmatisation** (verb then noun) normalises "crashing → crash", "issues → issue" cheaply, without a costly POS tagger.

3.2 Vectorisation ([src/vectorization.py](#))

Strategy	Representation	Notes
Bag-of-Words	n-gram counts (1-2-grams, $\leq 5\,000$ features)	sparse, non-negative
TF-IDF	counts re-weighted by inverse document frequency, sublinear TF	sparse, non-negative
Word2Vec	mean of 100-d embeddings trained on the training corpus	dense
BERT (optional)	mean-pooled contextual embeddings	requires <code>transformers + torch</code>

BERT is implemented in full (`BertVectorizer`) but needs the optional deep-learning stack and network access to the model hub. Where those are unavailable it is skipped automatically and the experiments use BoW / TF-IDF / Word2Vec.

3.3 Classifiers ([src/models.py](#))

Naive Bayes (`MultinomialNB` for sparse, `GaussianNB` for dense — chosen automatically), Decision Tree (with pre- and post-pruning in the grid), Random Forest, AdaBoost, and Logistic Regression (used to demonstrate **L2 regularisation** via the tuned `C`).

3.4 Evaluation ([src/evaluation.py](#))

Accuracy, **macro**-averaged precision/recall/F1 (so minority classes count equally), and confusion matrices. Macro-F1 is the tuning target.

3.5 Two engineering choices that keep it fast and correct

- **Preprocess once.** Cleaning is stateless, so running it a single time up front is exactly equivalent to running it inside every CV fold — but far cheaper.
- **Cache vectorisers** (`joblib.Memory`) against one fixed `StratifiedKFold` , so the expensive Word2Vec / TF-IDF fits are computed once per fold and reused across every hyper-parameter combination and classifier. The full study runs in ~ 130 s on 4 cores.

4. Experimental setup

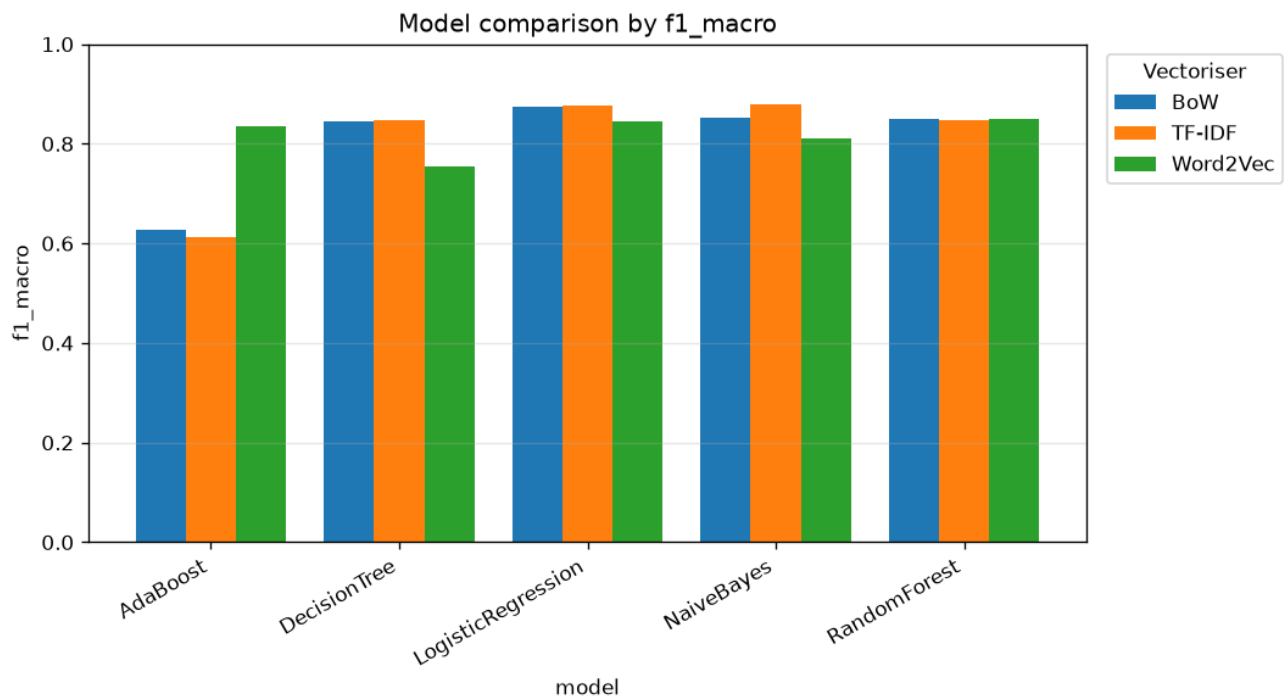
- **Split:** stratified 80 % train / 20 % test (`TEST_SIZE = 0.20`).

- **Cross-validation:** 5-fold `StratifiedKFold` on the training set.
- **Tuning:** `GridSearchCV`, scoring `f1_macro`, refit on the best configuration.
- **Reproducibility:** every stochastic component seeded from `RANDOM_STATE = 42` ;
Word2Vec uses a single worker for determinism.

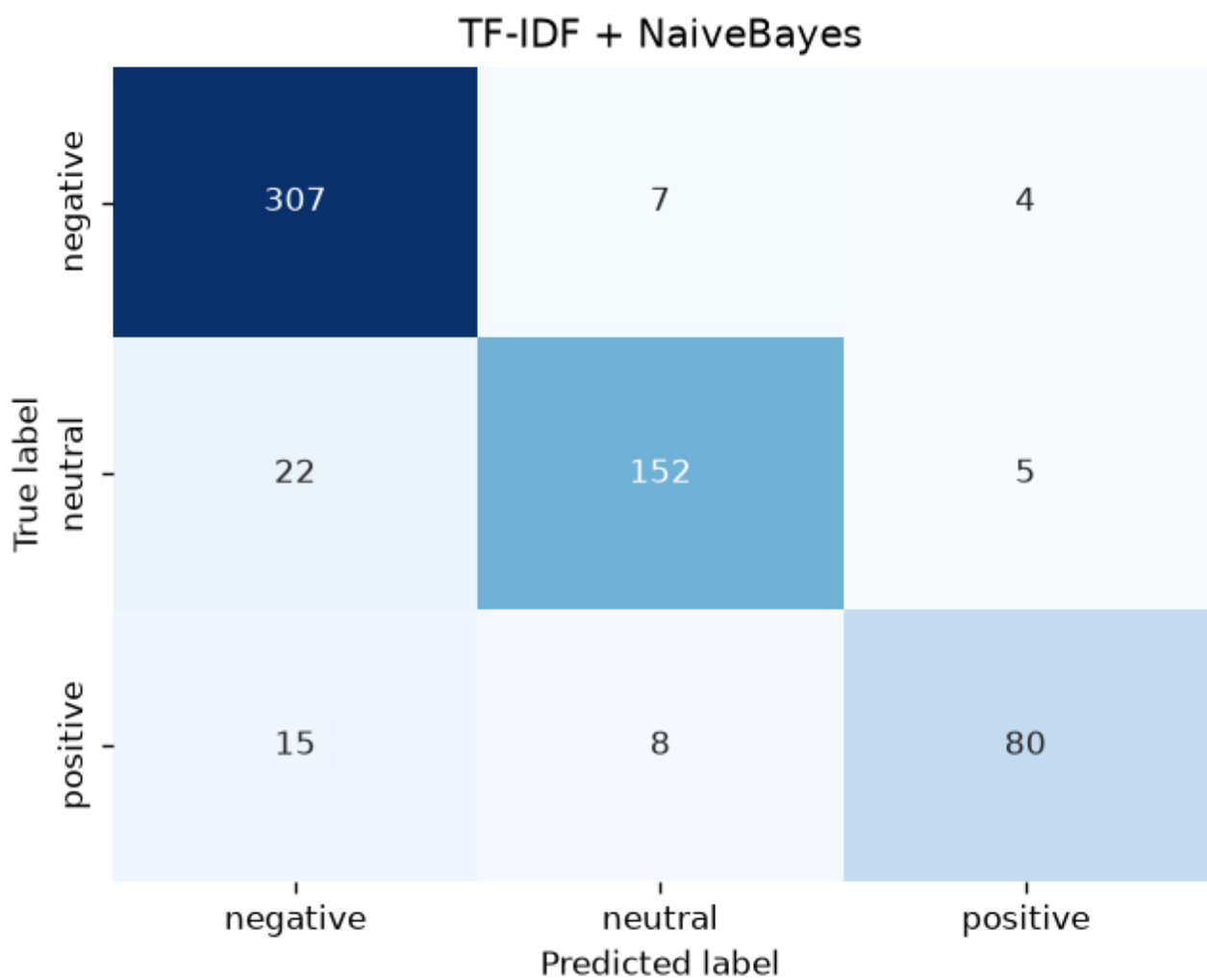
5. Model comparison

Test-set results (held-out 20 %), sorted by macro-F1:

Rank	Vectoriser	Classifier	CV F1	Accuracy	Precision	Recall	F1 (macro)
1	TF-IDF	Naive Bayes	0.850	0.898	0.900	0.864	0.880
2	TF-IDF	Logistic Regression	0.843	0.895	0.903	0.858	0.877
3	BoW	Logistic Regression	0.841	0.895	0.901	0.854	0.874
4	BoW	Naive Bayes	0.835	0.877	0.853	0.852	0.852
5	BoW	Random Forest	0.832	0.873	0.876	0.832	0.850
6	Word2Vec	Random Forest	0.810	0.870	0.867	0.835	0.849
7	TF-IDF	Decision Tree	0.802	0.870	0.871	0.832	0.849
8	TF-IDF	Random Forest	0.825	0.867	0.864	0.834	0.847
9	BoW	Decision Tree	0.814	0.867	0.869	0.828	0.845
10	Word2Vec	Logistic Regression	0.827	0.868	0.856	0.836	0.845
11	Word2Vec	AdaBoost	0.787	0.860	0.849	0.826	0.836
12	Word2Vec	Naive Bayes	0.775	0.838	0.803	0.824	0.811
13	Word2Vec	Decision Tree	0.737	0.795	0.763	0.750	0.756
14	BoW	AdaBoost	0.606	0.742	0.835	0.601	0.628
15	TF-IDF	AdaBoost	0.610	0.733	0.826	0.589	0.612



Confusion matrix of the best model (TF-IDF + Naive Bayes):



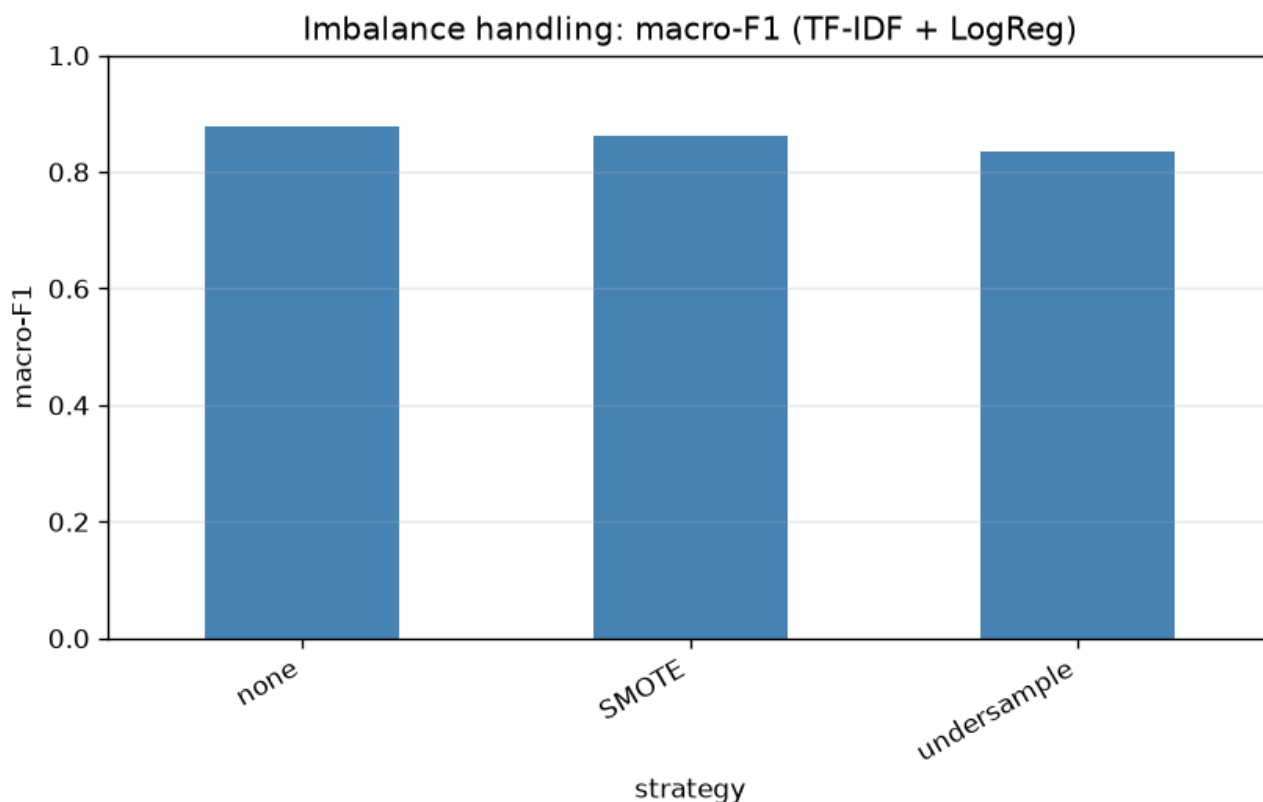
Reading the results

- **TF-IDF + Naive Bayes wins (F1 = 0.880).** For short, keyword-driven text the IDF re-weighting plus NB's strong independence assumption is hard to beat, and TF-IDF clearly improves NB over plain BoW (0.880 vs 0.852).
- **Logistic Regression is the most robust learner** — top-3 on TF-IDF/BoW and still strong on Word2Vec (0.845–0.877) — thanks to L2 regularisation.
- **AdaBoost collapses on sparse high-dimensional text (0.61–0.63)** because its depth-1 stumps can each test only a single word out of ~5 000; yet it recovers to 0.836 on dense 100-d Word2Vec, where each split is informative. A textbook illustration of matching the model to the feature geometry.
- **Word2Vec trails the sparse methods (≈ 0.81 –0.85):** embeddings trained on a small in-domain corpus blur the precise lexical cues that TF-IDF preserves, but they make tree/boosting models behave far better (dense, low-dimensional).

6. Class-imbalance handling

Fixed pipeline (TF-IDF + Logistic Regression); only the resampling strategy changes. Resampling is applied **inside an imblearn pipeline**, i.e. to the training folds only.

Strategy	Accuracy	Macro-F1	Recall neg	Recall neu	Recall pos
none (baseline)	0.895	0.877	0.969	0.838	0.767
SMOTE	0.885	0.863	0.931	0.860	0.786
under-sampling	0.860	0.834	0.887	0.855	0.786

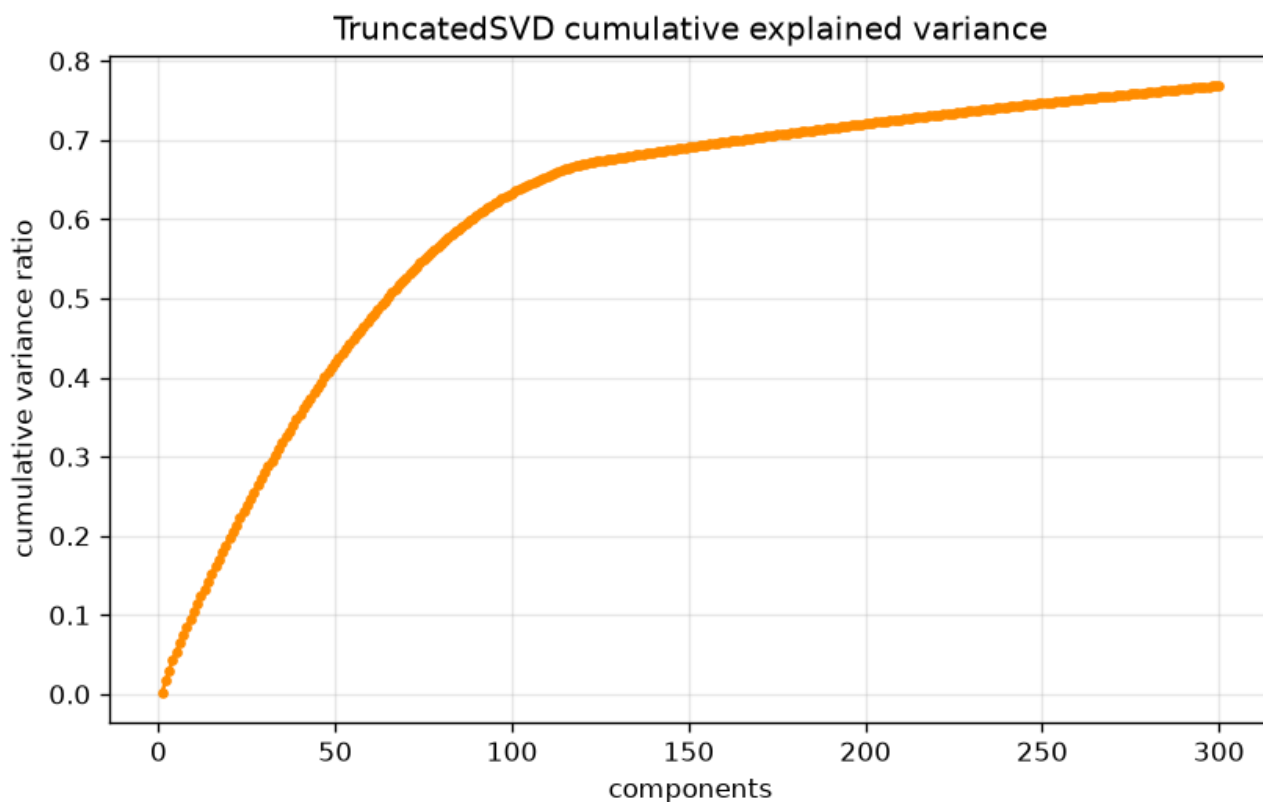


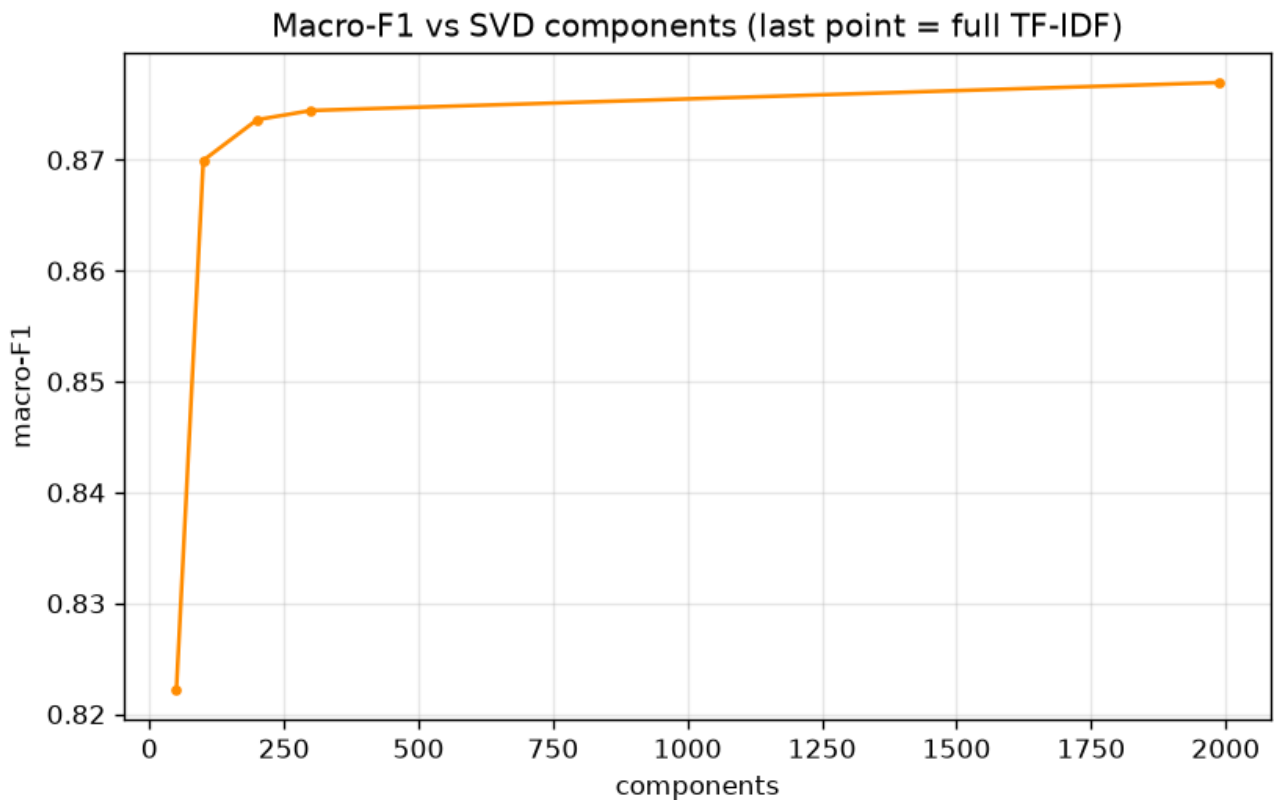
Interpretation. Both resampling methods raise minority-class recall — `positive` 0.767 → 0.786 and `neutral` 0.838 → 0.860 — but at the cost of `negative` recall (0.969 → 0.931 with SMOTE, → 0.887 under-sampling), so overall macro-F1 slips slightly. This is the expected behaviour: resampling **redistributes errors toward the minority classes** rather than creating free accuracy. SMOTE dominates under-sampling because it synthesises new minority points instead of discarding majority data. On this dataset the regularised baseline already handles the imbalance well, so we keep it; on a harder imbalance SMOTE would be the choice.

7. Dimensionality reduction (PCA / TruncatedSVD)

The TF-IDF matrix is sparse, so plain PCA (which centres the data) would densify it wastefully. We use **TruncatedSVD** — the PCA-equivalent for sparse text (a.k.a. LSA) — and measure downstream macro-F1 (TF-IDF + Logistic Regression).

Components	Explained variance	Macro-F1
50	41.7 %	0.822
100	63.3 %	0.870
200	71.9 %	0.874
300	76.9 %	0.874
1 987 (full)	100 %	0.877



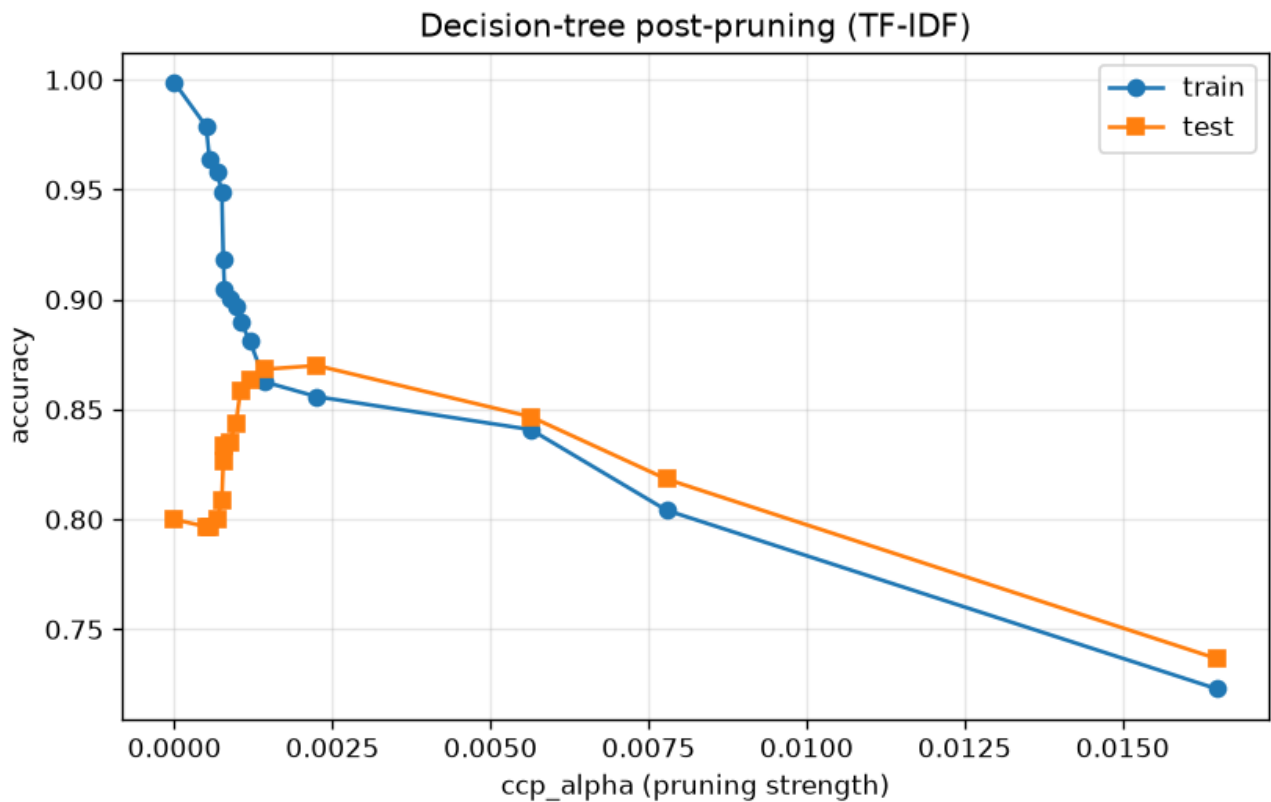


Interpretation. Just **200 components (~10 % of the 1 987 features)** recover **0.874 of the 0.877** full-feature macro-F1 — a 10× compression for a 0.003 drop. Dimensionality reduction is therefore an excellent speed/memory trade-off here, and would be essential before feeding sparse text to a dense model such as gradient boosting (see §9).

8. Decision-tree pruning

A single Decision Tree on TF-IDF features, traced along its **cost-complexity post-pruning path** (`ccp_alpha`):

<code>ccp_alpha</code>	# nodes	Train acc	Test acc
0.0000 (unpruned)	707	0.999	0.800
0.0010	147	0.890	0.858
0.0014	81	0.863	0.868
0.0023	65	0.856	0.870
0.0078	35	0.804	0.818
0.0165 (over-pruned)	19	0.723	0.737

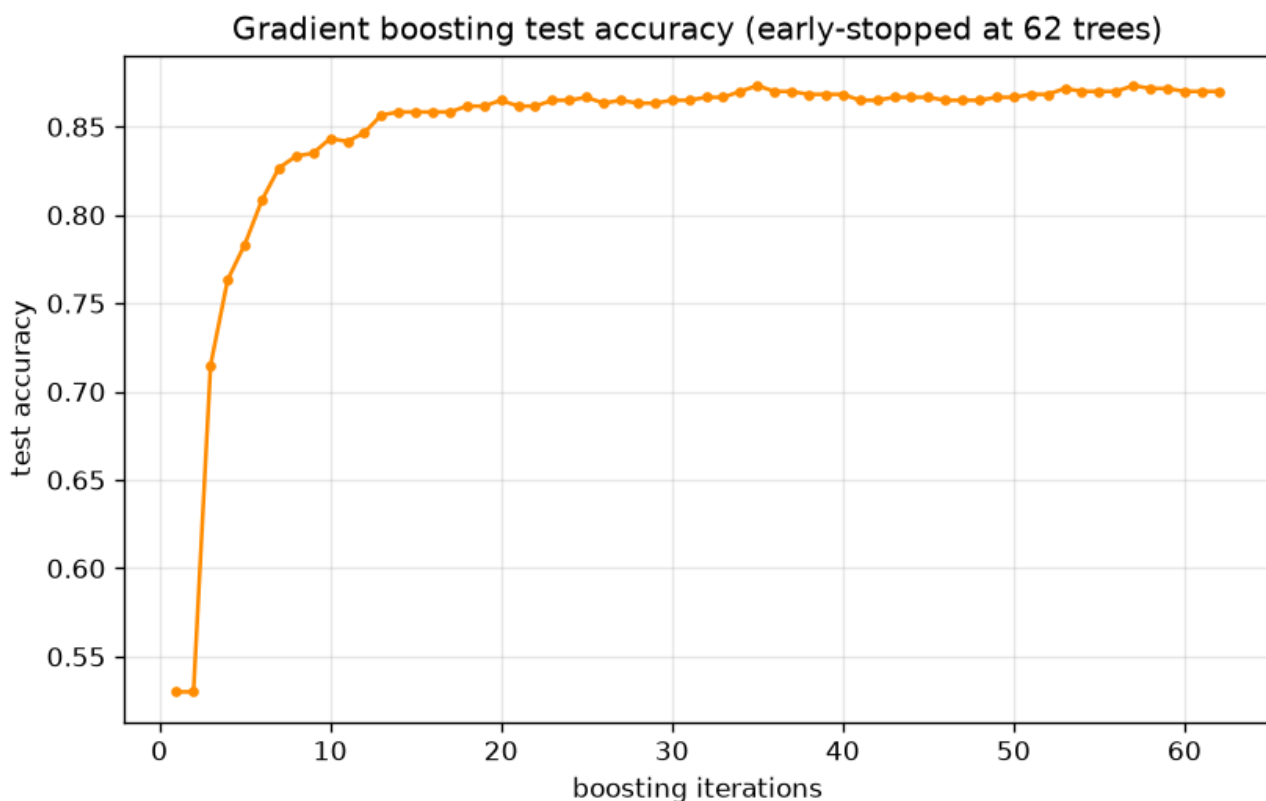


Interpretation. The unpruned tree memorises the training set (train 0.999, test 0.800 — a 0.20 generalisation gap). Increasing `ccp_alpha` shrinks the tree from 707 to ~65 nodes while **raising test accuracy to 0.870** and closing the gap; pruning too hard (19 nodes) underfits. Pre-pruning (`max_depth` , `min_samples_leaf`) is also tuned in the main grid (§5). This is the classic bias-variance trade-off made visible.

9. Early stopping (gradient boosting)

Gradient boosting (`GradientBoostingClassifier`) on TF-IDF → TruncatedSVD(100) features, with a generous ceiling of 500 trees and `n_iter_no_change = 10` monitoring a validation slice.

Max trees	Trees actually used	Test accuracy	Test macro-F1
500	62	0.870	0.843



Interpretation. Early stopping halted training after **62 of 500** trees once validation performance plateaued — an $\approx 8\times$ **reduction** in training cost with no loss of test quality, and protection against the over-fitting that more trees would bring. (The SVD reduction from §7 is what makes boosting on text efficient.)

10. Discussion: justification of techniques and trade-offs

Decision	Why	Trade-off
TF-IDF over BoW	IDF down-weights ubiquitous terms; best single result	marginally costlier to fit
Keep negation stop-words	preserves sentiment-flipping cues	slightly larger vocabulary
Multinomial vs Gaussian NB by feature type	NB assumptions must match the data	none (handled automatically)
Logistic Regression with L2	strong, robust, regularised baseline	linear decision boundary
TruncatedSVD not PCA	avoids densifying sparse TF-IDF	loses exact feature interpretability
Macro-F1 as the metric	weights minority classes equally on an imbalanced problem	hides raw accuracy (reported alongside)
Resampling inside <code>imblearn</code> pipeline	prevents leakage of synthetic samples into validation	extra pipeline plumbing
Preprocess once + cache vectorisers	~ 130 s for the whole study	a few hundred MB of disk cache

Headline findings. (1) Simple linear / probabilistic models on TF-IDF are the sweet spot for short issue text. (2) Boosting needs dense, low-dimensional features to work — never raw sparse n-grams. (3) Resampling trades majority recall for minority recall rather than improving everything at once. (4) Both pruning and early stopping convincingly curb overfitting.

11. Challenges faced

- **No downloadable labelled corpus** in the sandbox → built a controlled, reproducible synthetic generator instead, and had to calibrate its difficulty (ambiguity, mixed signals, label noise) so the comparison was informative rather than saturated at a single score.
 - **MultinomialNB rejects negative features** → automatic switch to GaussianNB for dense embeddings / reduced features.
 - **Run-time of nested tuning** across 15 vectoriser×model combinations → solved with stateless-preprocessing-once + a shared `joblib` vectoriser cache.
 - **BERT weights unreachable** behind the network policy → a complete BERT vectoriser is provided but gracefully skipped, with the other three vectorisers carrying the comparison.
-

12. Future work

- Run on a **real** corpus (GitHub/JIRA issues, App-Store reviews) once API access is available; the pipeline already accepts any `text,label` CSV.
 - Enable the **BERT** path and add a fine-tuned transformer for comparison.
 - Try **class-weighting** as a leakage-free alternative to resampling, and cost-sensitive thresholds for the minority class.
 - Add **calibrated probabilities** and per-class precision/recall curves for triage use-cases (e.g. auto-escalating angry reports).
-

13. Reproducibility

```
pip install -r requirements.txt
python main.py                # full study (~130 s on 4 cores), writes results/
```

Every figure and table in this report is regenerated under [results/](#) by that single command, from the seed `RANDOM_STATE = 42`.